난해한 언어 모델

Subham Sekhar Sahoo*[1] Zhihan Yang*[2] Yash Akhauri†[1] Johnna Liu†[1] Deepansha Singh†[1] Zhoujun Cheng†[3] Zhengzhong Liu[3] Eric Xing[3] John Thickstun[2] Arash Vahdat[4] *공동 제1저자 †공동 제2저자 [1] 코넬 테크 [2] 코넬 대학교 [3]MBZUAI [4] NVIDIA {ssahoo, zhihany}@cs.cornell.edu

초록

확산 기반 언어 모델은 병렬 및 제어 가능한 생성을 가능하게 함으로써 자기회귀(AR) 모델에 대한 설득력 있는 대안을 제공합니다. 이 모델군 중에서 마스크 확산 모델(MDM)은 가장 강력한 성능을 달성하지만, perplexity 면에서는 여전히 AR 모델에 미치지 못하며 핵심 추론 시간 효율성 기능, 특히 KV 캐싱이 부족합니다. 본 연구에서는 AR 및 MDM 패러다임을 융합하여 perplexity 측면에서 매끄러운 보간을 가능하게 하면서 각 모델의 한계를 극복하는 새로운 모델군인 Eso-LM을 소개합니다. Eso-LM은 표준 언어 모델링 벤치마크에서 새로운 최첨단 성능을 확립합니다. 결정적으로, 우리는 병렬 생성을 유지하면서 MDM에 KV 캐싱을 도입한 최초의 사례로, 추론 효율성을 크게 향상시킵니다. 최적화된 샘플링 스케줄과 결합된 우리 방법은 표준 MDM보다 최대 65배, 이전의 준자기회귀(semi-autoregressive) 접근 방식보다 4배 빠른 추론을 달성합니다. 코드와 모델 체크포인트는 프로젝트 페이지에서 제공됩니다.

https://s-sahoo.com/Eso-LMs

1 서론

우리는 언어 모델링의 중요한 전환점에 서 있습니다. 패러다임의 변화가 진행 중입니다. 오랫동안 표준으로 여겨졌던 자기회귀(AR) 언어 모델이 이제 표준 언어 생성에서 확산 언어 모델과 경쟁하고 있습니다. 최근 연구 *[26, 29]*에 따르면 마스크 확산 모델(MDM)이 소규모 언어 벤치마크에서 AR 모델과의 격차를 좁히고 있으며, 분자 생성 [28, 11] 및 그래프 생성 *

이러한 결과는 MDM을 AR 모델의 설득력 있는 대안으로 만듭니다. 그러나 MDM은 두 가지 주요 한계점을 가지고 있습니다. (1) 추론 속도: 병렬 생성을 지원함에도 불구하고, MDM은 KV 캐싱 부족(채팅 시스템과 같은 실시간 애플리케이션에 필수적인 최적화)으로 인해 실제로 AR 모델보다 훨씬 느립니다. (2) 생성 품질: MDM은 더 복잡한 언어 모델링 작업 *

최근 제안된 BD3-LM *

!

캐싱: KV 캐싱은 블록 간에는 가능하지만, 블록 내 확산은 여전히 KV 지원이 부족하여 전반적인 속도 향상을 제한합니다.

이러한 문제들을 해결하기 위해 우리는 자기회귀 및 마스크 확산 접근 방식을 융합한 새로운 언어 모델링 패러다임을 제안합니다. 우리 모델은 AR 및 MDM 목표의 조합인 하이브리드 손실로 훈련되어 perplexity 측면에서 두 패러다임 간에 원활한 보간을 가능하게 합니다. 이를 위해서는 두 가지 핵심 혁신이 필요합니다. (1) AR 및 MDM 스타일의 생성을 모두 지원하기 위한 노이즈 제거 트랜스포머의 수정된 어텐션 메커니즘. (2) MDM에서 이전에 사용할 수 없었던 기능인 확산 단계 내에서 KV 캐싱을 가능하게 하는 새로운 훈련 및 샘플링 절차. 이 하이브리드 디자인의 비전통적인 특성 때문에 우리 방법을 Esoteric Language Models (Eso-LMs)라고 명명합니다.

우리의 방법은 언어 모델링 벤치마크에서 확산 모델 중 새로운 최첨단 성능을 달성하며, AR 및 MDM perplexity 간의 보간을 정밀하게 제어할 수 있게 합니다. 이전 접근 방식과 달리, 확산 중에 KV 캐싱을 지원하여 추론 속도를 현저히 높입니다. 요약하자면, 우리의 기여는 다음과 같습니다. (1) AR 및 MDM 패러다임을 융합하고 이전 하이브리드 접근 방

식인 BD3-LM을 능가하는 새로운 언어 모델링 프레임워크를 제안합니다. (2) 병렬 생성을 유지하면서 MDM에 KV 캐싱을 가능하게 한 최초의 접근 방식이며, 표준 MDM보다 최대 65×배, KV 캐싱이 적용된 준자기회귀(semi-autoregressive) 기준선보다 3−4×배 빠른 추론을 달성합니다 (5.2절). (3) BD3-LM이 낮은 샘플링 단계에서 성능이 저하되는 반면, 우리 방법은 낮은 NFE 체제에서는 MDM과, 높은 NFE 체제에서는 AR과 경쟁력이 있음을 보여줍니다 (5.3절). (4) 우리의 결과는 이산 확산에 대한 새로운 최첨단 perplexity를 확립하여 자기회귀 모델과의 격차를 좁힙니다 (5.1절).

2 배경

표기법 K개의 값을 가질 수 있는 스칼라 이산 확률 변수를 '원-핫' 열 벡터로 표현하고, $\mathcal{V}\in\{\mathbf{x}\in\{0,1\}^K:\Sigma_{i=1}^K\mathbf{x}_i=1\}$을 이러한 모든 벡터의 집합으로 정의합니다. $\mathbf{m}\in\mathcal{V}$를 $\mathbf{m}_K=1$인 특별한 마스크 벡터로 둡니다. $\text{Cat}(\cdot;\pi)$를 K개의 클래스에 대한 범주형 분포로 정의하며, 확률은 $\pi\in\Delta^K$으로 주어지고, 여기서 Δ^K은 K심플렉스를 나타냅니다. 또한, $\langle\mathbf{a},\mathbf{b}\rangle$는 벡터 $\mathbf{a}$와 $\mathbf{b}$의 내적을 나타냅니다. 우리는 정렬된 집합(튜플)을 나타내기 위해 괄호 ()를 사용하고, 정렬되지 않은 집합을 나타내기 위해 중괄호 {}를 사용합니다. $|A|$는 집합 A의 원소 수를 나타냅니다.

MDM은 두 가지 두드러진 순서를 특징으로 합니다: 시퀀스 순서와 노이즈 제거 순서. 우리는 이러한 순서 간의 관계를 설명하기 위해 순열 σ을 사용합니다. $\mathcal{P}^L$을 $[L]=\{1,\cdots,L\}$의 모든 순열 집합으로 나타냅니다. $\sigma\in\mathcal{P}^L$는 정렬된 집합(튜플)이며 전단사 함수로도 기능합니다: $\sigma(l)$는 lth 위치의 항목이고, σ는 $\sigma^{-1}(i)$ 항목의 위치입니다. 예를 들어, i는 σ의 순서를 나타내며, (2,4,1,3)는 위치 2를 해당 항목 4에 매핑하고, {1,2,3,4}는 항목 4를 해당 위치 2에 매핑합니다.

$\mathbf{x}\in\mathcal{V}^L$는 마스크 토큰이 없는 L 길이의 시퀀스를 나타내고, $\mathbf{x}^\ell$는 ℓth의 $\mathbf{x}$번째 항목을 나타냅니다. 우리는 '토큰 인덱스'라는 용어를 원본 순서에서 토큰의 위치를 지칭하는 데 사용합니다. 예를 들어, $\mathbf{x}^l$의 토큰 인덱스는 l입니다. $(\mathbf{z}_t)_{t\in[0,1]}\in\mathcal{V}^L$은 마스크 토큰을 포함할 수 있는 L 길이의 시퀀스를 나타냅니다. $\mathcal{M}(\mathbf{z}_t)=\{\ell|\mathbf{z}_t^\ell=\mathbf{m}\}$는 $\mathbf{z}_t$ 내 마스크 토큰의 토큰 인덱스를 나타내고, $\mathcal{C}(\mathbf{z}_t)=\{\ell|\mathbf{z}_t^\ell\neq\mathbf{m}\}$은 $\mathbf{z}_t$ 내 클린 토큰의 토큰 인덱스를 나타냅니다.

$\oplus:\mathcal{V}^m\times\mathcal{V}^n\to\mathcal{V}^{m+n}$를 $\mathbf{x}=(\mathbf{x}^1,\mathbf{x}^2,\cdots,\mathbf{x}^m)$ 및 $\mathbf{z}=(\mathbf{z}^1,\mathbf{z}^2,\cdots,\mathbf{z}^n)$ 길이의 두 시퀀스 m 및 n에 대한 연결 연산자로 둡니다. 연결된 시퀀스 $\mathbf{x}\oplus\mathbf{z}$가 트랜스포머에 입력될 때, $\mathbf{x}$ 및 $\mathbf{z}$은 개별적으로 트랜스포머에 입력되었을 때와 동일한 위치 임베딩을 가집니다. $\odot:\mathcal{V}^m\times\mathcal{V}^n\to\mathcal{V}^m$를 치환 연산자로 둡니다. 임의의 $\mathbf{z}\in\mathcal{V}^m$ 및 $\mathbf{x}\in\mathcal{V}^n$ $(m>n)$에 대해, 출력 $\mathbf{y}=\mathbf{z}\odot\mathbf{x}$는 다음과 같습니다: $\mathbf{y}^{1:n}=\mathbf{x}$ 및 $\mathbf{y}^{n+1:m}=\mathbf{z}^{n+1:m}$.

2.1 자기회귀 모델

$\mathbf{x}\sim q_{\text{data}}$ 내의 시퀀스 $\mathcal{V}^L$가 주어졌을 때, 자기회귀(AR) 모델은 다음 결합 분포의 인수분해를 정의합니다: $\log p_\theta(\mathbf{x})=\Sigma_{t=1}^L\log p_\theta(\mathbf{x}^\ell|\mathbf{x}^{<\ell})$. 여기서 모델 p_θ는 일반적으로 인과적 트랜스포머 *

2.2 마스크 확산 모델

확산 모델은 순방향 손상 과정 q을 역전시키는 방법을 학습합니다. 이 과정은 $\mathcal{V}^L$ 내의 깨끗한 데이터 $\mathbf{x}\sim q_{\text{data}}$를 $t\in[0,1]$에 대한 잠재 변수 시퀀스 $\mathbf{z}_t$로 변환하며, 각 잠재 변수는 $\mathbf{x}$의 점점 더 노이즈가 많은 버전을 나타냅니다 [31, 32, 8]. MDM *[26, 30, 20]*에서, 순방향 마스킹 과정 q은 시퀀스 $\mathbf{x}$에 걸쳐 독립적으로 인수분해됩니다. 즉, $q_t(.|\mathbf{x})=\Pi^\ell q_t(.|\mathbf{x}^\ell)$이고 각 토큰 $\mathbf{x}^l$는 고정된 목표 분포 $\text{Cat}(.;\mathbf{m})$와 점진적으로 보간됩니다. 시간 t에서의 $\mathbf{z}_t^\ell\sim q_t(.|\mathbf{x}^\ell)$의 주변 확률은 다음과 같습니다.

$$q_t(.|\mathbf{x}^\ell)=\text{Cat}(.;\alpha_t\mathbf{x}^\ell+(1-\alpha_t)\mathbf{m}),\quad (1)$$

여기서 $\alpha_t\in[0,1]$는 t에 대해 엄격히 감소하는 함수이며 $\alpha_0\approx1$ 및 $\alpha_1\approx0$입니다. *Sahoo et al.

[$q_{s|t}(.|₩mathbf{z}_t^{₩ell}, ₩mathbf{x}^{₩ell}) = ₩begin{cases} ₩text{Cat}(.; ₩mathbf{z}_t^{₩ell}) & ₩mathbf{z}_t^{₩ell} ₩neq ₩mathbf{m}, \\ ₩text{Cat}₩left(:{₩frac{(1-₩alpha_s)₩mathbf{m} + ({₩alpha_s - ₩alpha_t})₩mathbf{x}^{₩ell}}{1-₩alpha_t}}₩right) & ₩mathbf{z}_t^{₩ell} = ₩mathbf{m}. ₩end{cases}]$ (2)

노이즈 제거 모델 $\mathbf{x}\theta:\mathcal{V}^L \to (\Delta^K)^L$가 주어졌을 때, 시퀀스 $\mathbf{z}_s$에 대한 역 마스크 해제 과정 $p\theta(.|\mathbf{z}t)s|t$은 다음과 같이 매개변수화됩니다.

$p\theta(.|\mathbf{z}t)s|t = \Pi\ell^L p\theta\ell(.|\mathbf{z}t)s|t = \Pi t^L q s|t\ell(.|\mathbf{z}t\ell, \mathbf{x}\ell = \mathbf{x}\theta\ell(\mathbf{z}t))$. (3)

*Sahoo et al.

$\mathscr{L}MDM(\mathbf{x}) = \mathbb{E}q,t\sim[0,1]\alpha t' 1-\alpha t \Sigma\ell \in \mathscr{M}(\mathbf{z}t)\log\langle \mathbf{x}\theta\ell(\mathbf{z}t), \mathbf{x}\ell\rangle$, (4)

이는 마스크된 토큰 $\mathscr{M}(\mathbf{z}t)$에 대해서만 계산되는 마스크 언어 모델링 손실 *

길이 L의 시퀀스를 생성하기 위해 역확산 프로세스는 $\mathbf{z}t=1\ell = \mathbf{m}$는 $\ell=1,\cdots,L$입니다. 이 프로세스는 T 단계 동안 진행되며, 각 $\mathbf{z}s\ell$는 (3)에 정의된 대로 $p\theta(.|\mathbf{z}t)s|t$에서 독립적으로 샘플링됩니다. 일단 위치가 마스크 해제되면 고정된 상태로 유지됩니다. 여러 토큰을 병렬로 노이즈 제거할 수 있으므로, 총 네트워크 함수 평가(NFE) 횟수는 L보다 적을 수 있어 더 빠른 생성을 가능하게 합니다. 그러나 각 NFE는 $\mathbf{x}\theta(\mathbf{z}t)$의 양방향 트랜스포머로 인해 비용이 많이 듭니다.

2.3 블록 노이즈 제거 확산 이산 언어 모델

블록 노이즈 제거 확산 이산 언어 모델(BD3-LM) *

BD3-LM은 AR 모델과 MDM 사이를 보간합니다. BD3-LM은 $\mathbf{x}$의 토큰을 B=L/L′을 사용하여 L′개의 연속 토큰으로 구성된 B개 블록으로 그룹화하며, 여기서 B는 정수라고 가정합니다. $\mathbf{x}$에 대한 우도(likelihood)는 블록에 대해 자기회귀적으로 $-\log p\theta(\mathbf{x}) = -\Sigma b=1B \log p\theta(\mathbf{x}b|\mathbf{x}<b) \leq \Sigma b=1B \mathscr{L}MDM(\mathbf{x}b;\mathbf{x}<b)$으로 인수분해되며, 여기서 $p\theta(\mathbf{x}b|\mathbf{x}<b)$은 조건부 MDM이고 $\mathscr{L}MDM(\mathbf{x}b;\mathbf{x}<b)$은 MDLM에 대한 NELBO로 (4)에 정의되어 있으며 블록 전체에 순차적으로 적용됩니다.

3 Esoteric 언어 모델

우리는 언어 모델링을 위한 새로운 패러다임인 Esoteric 언어 모델(Eso-LM)을 제안하며, 이는 AR 모델과 MDM의 공생적 조합을 이룹니다. 우리의 접근 방식에는 두 가지 주요 구성 요소가 있습니다. (1) 순차적(AR 방식) 및 병렬(MDM 방식) 생성을 모두 지원하는 모델 패밀리를 소개합니다. (2) 섹션 4에서 이러한 하이브리드 생성을 가능하게 하는 트랜스포머 *

현재 AR 모델은 언어 모델링 성능에서 선두를 달리고 있지만, 토큰을 순차적으로 생성하여 추론 속도가 느립니다. 이와 대조적으로 MDM은 여러 토큰을 병렬로 생성하며 더 제어 가능하지만 [29, 19], 일반적으로 AR 모델보다 더 높은 혼란도(perplexity)를 산출합니다 [26, 27]. 핵심 질문은: 이들의 강점을 결합할 수 있는가?입니다. 우리는 일부 토큰은 MDM을 통해 병렬로 생성하고 나머지는 왼쪽에서 오른쪽으로 순차적으로 생성하는 하이브리드 접근 방식을 도입합니다. 이는 두 가지 주요 질문을 제기합니다. 첫째, 이러한 생성 모델의 우도를 계산할 수 있는가? 우리는 Eso-LM이 실제 우도에 대한 원리적인 경계를 허용함을 보여줍니다. 둘째, 트랜스포머의 어텐션 메커니즘을 두 가지 생성 스타일을 모두 지원하도록 어떻게 조정할 수 있는가? 훈련 중, AR 모델에서는 깨끗한 시퀀스에 인과적 어텐션이 강제되는 반면, MDM은 마스크된 토큰과 마스크되지 않은 토큰 모두에 대한 양방향 어텐션에 의존합니다. 섹션 4에서 우리는 트랜스포머를 위한 맞춤형 어텐션 메커니즘을 설계하여 이 두 가지 distinct 아키텍처를 조화시킵니다. 또한, 우리는 어텐션 행렬에 희소성을 도입하여 MDM을 위한 KV-캐싱을 가능하게 하며, 이는 장문맥 생성 속도를 크게 향상시킵니다.

3.1 자기회귀 모델과 마스크 확산 융합

데이터 분포에서 샘플인 $\mathcal{V}^L$ 내의 $\mathbf{x}$~qdata($\mathbf{x}$)과, θ으로 매개변수화된 모델 분포인 pθ라고 가정합니다. Eso-LM은 pθ를 두 가지 구성 요소, 즉 AR 모델 pθAR와 MDM pθMDM으로 분해합니다. MDM은 부분적으로 마스크된 시퀀스 $\mathbf{z}_0 \in \mathcal{V}^L$~pθMDM($\mathbf{z}_0$)을 생성하고, AR 모델은 나머지 마스크 해제 단계를 자기회귀적으로 왼쪽에서 오른쪽으로 완료합니다: pθAR($\mathbf{x}|\mathbf{z}_0$). 이러한 하이브리드 생성 프로세스의 한계 우도는 다음과 같습니다.

$$p_\theta(\mathbf{x}) = \Sigma_{\mathbf{z}_0 \in \mathcal{V}^L} p_\theta^{AR}(\mathbf{x}|\mathbf{z}_0) p_\theta^{MDM}(\mathbf{z}_0). \quad (5)$$

이 합계는 다루기 어렵지만, 사후 분포 q($\mathbf{z}_0|\mathbf{x}$) *

$$-\log p_\theta(\mathbf{x}) \leq -\mathbb{E}_{\mathbf{z}_0 \sim q_0(\cdot|\mathbf{x})}[\log p_\theta^{AR}(\mathbf{x}|\mathbf{z}_0)] + D_{KL}(q_0(\mathbf{z}_0|\mathbf{x}) \| p_\theta^{MDM}(\mathbf{z}_0)) \quad (6)$$
$$= -\mathbb{E}_{\mathbf{z}_0 \sim q_0(\cdot|\mathbf{x})}[\Sigma_{\ell \in \mathcal{M}(\mathbf{z}_0)} \log p_\theta^{AR}(\mathbf{x}_\ell|\mathbf{z}_0, \mathbf{x}_{<\ell})] + D_{KL}(q_0(\mathbf{z}_0|\mathbf{x}) \| p_\theta^{MDM}(\mathbf{z}_0)).$$

여기서 pθAR($\mathbf{x}|\mathbf{z}_0$)은 마스크된 위치 $\ell \in \mathcal{M}(\mathbf{z}_0)$에 대한 결합 AR 우도이며, $\mathbf{z}_0$의 깨끗한 토큰에 조건화됩니다. AR의 경우, 노이즈 제거 네트워크 $\mathbf{x}_\theta: \mathcal{V}^L \to (\Delta^K)^L$은 입력 $\mathbf{z}_0 \odot \mathbf{x}_{<\ell}$에서 작동하며, 여기서 치환 연산자 ⊙는 $\mathbf{z}_0$의 첫 $l-1$개 토큰을 $\mathbf{x}_{<\ell}$으로 대체합니다. 각 $\ell \in \mathcal{M}(\mathbf{z}_0)$에 대해 $\mathbf{x}_\theta^\ell(\mathbf{z}_0 \odot \mathbf{x}_{<\ell})$은 $\mathbf{x}_{<\ell}$ 및 $\mathbf{z}_0$이 주어진 깨끗한 토큰 $\mathbf{x}_\ell$의 분포를 근사하며, 이는 위치 ℓ를 넘어선 깨끗한 토큰을 포함할 수 있습니다. 부록 B.1에서 우리는 KL 항을 분석하고 NELBO가 다음과 같음을 보여줍니다.

$$\mathcal{L}_{NELBO}(\mathbf{x}) = \mathbb{E}_{\mathbf{z}_0 \sim q_0}[-\Sigma_{\ell \in \mathcal{M}(\mathbf{z}_0)} \log \langle \mathbf{x}_\theta^\ell(\mathbf{z}_0 \odot \mathbf{x}_{<\ell}), \mathbf{x}_\ell \rangle]}_{\text{AR loss}} + \int_{t=0}^{t=1} \alpha_t' (1-\alpha_t) \mathbb{E}_{\mathbf{z}_t \sim q_t}[\Sigma_{\ell \in \mathcal{M}(\mathbf{z}_t)} \log \langle \mathbf{x}_\theta^\ell(\mathbf{z}_t), \mathbf{x}_\ell \rangle]}_{\text{M} \quad (7)$$

AR과 확산 사이의 보간

α0=1일 때, 사후 샘플은 $\mathbf{z}_0 = \mathbf{x}$이고 모든 토큰은 MDM에 의해 생성됩니다. 따라서 (7)에서 AR 손실은 0이고 $\mathcal{L}_{NELBO}$는 MDM 손실로 축소됩니다. 반대로 α0=0일 때, 모든 토큰은 마스크되고 MDM 손실은 사라지며 $\mathcal{L}_{NELBO}$는 AR 손실로 축소됩니다. 따라서 Eso-LM은 하이퍼파라미터 α0에 의해 제어되는 AR 및 MDM 패러다임 사이를 보간합니다.

3.2 샘플링

(5)에 정의된 2단계 샘플링을 수행합니다. pθ($\mathbf{x}$)에서 샘플링하기 위해 먼저 부분적으로 마스크된 시퀀스 $\mathbf{z}_0$~pθMDM을 그린 다음 pθAR를 사용하여 이를 완성합니다.

확산 단계

완전히 마스크된 시퀀스 $\mathbf{z}_{t=1} = \mathbf{m}^{1:L}$에서 시작하는 표준 조상 샘플링(2.2절)은 각 단계에서 마스크된 토큰의 부분집합만 업데이트하면서 전체 시퀀스에 대한 전방 전달을 수행하여 FLOP를 낭비합니다. 효율성을 높이기 위해, 우리는 표준 샘플링 및 훈련 절차에 두 가지 주요 변경 사항을 제안합니다. 샘플링 중에 확산 노이즈 제거 스케줄 $\mathcal{S}^{MDM} = (S^1, \cdots, S^{1/T})$을 미리 계산하며, 여기서 S^t는 확산 단계 t에서 노이즈 제거될 마스크 토큰 인덱스 집합입니다. 전체 시퀀스를 처리하는 대신, 부분 시퀀스 $\{\mathbf{z}_t^\ell | \ell \in \mathcal{C}(\mathbf{z}_t) \cup S^t\}$에 대해서만 전방 전달을 수행합니다. 즉, 깨끗한 토큰과 노이즈 제거가 예정된 토큰에 대해서만 수행하여 특히 긴 시퀀스에서 계산을 상당히 줄입니다. 이 샘플링 방식을 맞추기 위해, 모델은 4절에서 논의된 바와 같이 훈련 중 마스크 토큰에 대한 양방향 어텐션을 피해야 합니다. 이 접근 방식은 또한 4.2절에서 논의된 바와 같이 확산 중 효율적인 KV-캐싱을 가능하게 합니다.

순차 단계

최종 출력 $\mathbf{x}$~을 $\mathbf{z}_0$로 초기화합니다. 그런 다음 AR 모델은 순차적 노이즈 제거 스케줄 $\mathcal{S}^{AR} = ((i)|i \in \mathcal{M}(\mathbf{z}_0))$를 사용하여 마스크 토큰을 왼쪽에서 오른쪽으로 채워 넣습니다. 이때 단일 요소 집합은 그 유일한 요소에 따라 오름차순으로 정렬되

어야 합니다. 표준 AR 디코딩과 달리, 각 $\mathbf{x}$~ℓ은 완전히 깨끗한 토큰으로 구성된 왼쪽 컨텍스트와 $\mathbf{z}0$에 있는 오른쪽 깨끗한 토큰 모두에 조건화되어 더 풍부한 생성을 가능하게 합니다. 우리는 오른쪽 마스크 토큰에 대한 평가를 건너뛰어 불필요한 계산을 줄입니다. 순차 단계는 자연스럽게 KV 캐싱을 지원합니다.

우리는 통일된 노이즈 제거 스케줄을 $\mathcal{S}$=$\mathcal{S}$MDM∪$\mathcal{S}$AR으로 표기하며, 이는 [L]을 분할하기 위해 두 샘플링 스케줄을 연결합니다. α0=1일 때, 모든 토큰은 확산에 의해 생성되므로 $\mathcal{S}$=$\mathcal{S}$MDM 및 $\mathcal{S}$AR=∅ (널 집합)입니다. α0=0일 때, 모든 토큰은 순차적으로 생성되므로 $\mathcal{S}$=$\mathcal{S}$AR 및 $\mathcal{S}$MDM=∅입니다. 전체 샘플링 알고리즘 및 예시는 부록 B.4를 참조하십시오.

4 공유 노이즈 제거 트랜스포머를 위한 어텐션 메커니즘 설계

이 섹션에서는 공유 트랜스포머 아키텍처를 사용하여 순차적(AR) 및 병렬(MDM) 생성 모드를 모두 지원하는 통합 모델링 프레임워크를 소개합니다. 우리의 핵심 기술적 기여는 인과적 어텐션과 1단계 시프트 디코딩을 요구하는 AR 모델과 양방향 어텐션에 의존하는 MDM 사이의 아키텍처 불일치를 해결하는 유연한 어텐션 메커니즘입니다. 이를 달성하기 위해, 우리는 표준 어텐션을 다음과 같이 조절하는 어텐션 바이어스 행렬 A∈{−∞,0}L′×L′을 도입합니다. 여기서 L′은 시퀀스 길이입니다.

₩textscSelf−Attention(Q,K,V,A)=softmax(QK′d+A)V

여기서 Q,K,V∈ℝL′×d는 쿼리, 키, 값 행렬을 나타냅니다. A의 항목은 정보 흐름을 제어합니다. Ai,j=0는 토큰 i에서 j로의 어텐션을 "허용"하고, Ai,j=−∞는 "차단"합니다. 이 메커니즘은 단일 트랜스포머가 필요에 따라 인과적 및 양방향 동작을 모두 모방할 수 있도록 합니다.

이 통일된 어텐션 방식을 기반으로 두 가지 변형인 Eso-LM (A)와 Eso-LM (B)를 제안합니다. Eso-LM (A)는 어텐션을 희소화하고 각 확산 단계에서 마스크 토큰의 부분집합에만 노이즈 제거 트랜스포머를 선택적으로 적용하여 계산을 줄이는 데 중점을 둡니다. Eso-LM (B)는 이 아이디어를 확장하여 마스크 토큰뿐만 아니라 깨끗한 토큰에도 인과적 마스킹을 도입하여 더 큰 속도 향상을 위한 KV-캐싱을 가능하게 합니다. (이로 인해 혼란도가 약간 저하될 수 있습니다.) 이 섹션의 나머지 부분에서는 이러한 통일을 가능하게 하는 어텐션 공식을 설명하고 확산 및 순차 단계 모두에서 이 두 변형 간의 차이점을 자세히 설명합니다.

확산 단계

표준 MDM은 샘플링 중에 두 가지 비효율성으로 인해 어려움을 겪습니다: (i) 부분 업데이트에 대해서도 전체 시퀀스 순방향 전달, (ii) 양방향으로 인해 단계 간 KV 캐싱 없음

!

img-2.jpeg

img-3.jpeg

어텐션. 섹션 3.2에서 간략하게 논의했듯이, 우리는 단계 t에서 {$\mathbf{z}$tili∈$\mathcal{C}$($\mathbf{z}$t)∪St}에 대해서만 $\mathbf{x}$R을 평가함으로써 (i)을 해결합니다. 이 가능성을 열기 위해, 우리는 AO-ARM

순차 단계 순차적 생성 중에 모델은 $\mathbf{z}$0~pθMDM(.)에 의해 생성된 마스크 토큰 ($\mathbf{z}$0i)t∈$\mathcal{M}$($\mathbf{z}$0)을 왼쪽에서 오른쪽으로 노이즈 제거해야 합니다. 이는 생성 중에는 간단하지만 훈련 중에는 어렵습니다. 각 토큰이 왼쪽 토큰에만 어텐션하는 표준 AR 모델과 달리, 각 마스크 토큰 $\mathbf{z}$0i는 (i) 자기 자신 (위치 임베딩용), (ii) 클린 왼쪽 문맥 $\mathbf{x}$<i ($\mathbf{z}$0<i의 모든 이전 마스크 토큰이 채워졌다고 가정), (iii) 그리고 오른쪽 문맥 {$\mathbf{x}$jlj∈$\mathcal{C}$($\mathbf{z}$0),j>i} 또는 이와 동등한 {$\mathbf{z}$0jlj∈$\mathcal{C}$($\mathbf{z}$0),j>i}의 클린 토큰에 어텐션해야 합니다. 훈련 중에 (ii)를 처리하는 것은 간단하지 않습니다. 왜냐하면 $\mathbf{z}$0의 각 마스크 토큰은 자신의 왼쪽에 있는 토큰들이 이미 pθAR에 의해 노이즈 제거되었다고 가정하기 때문입니다. 이 동작을 시뮬레이션하기

위해, 우리는 훈련 중에 연결된 시퀀스 $\mathbf{z}0 \oplus \mathbf{x}$를 트랜스포머에 입력하고 모든 마스크 토큰에 대한 올바른 정보 흐름을 강제하는 사용자 정의 어텐션 마스크 A를 구성합니다. 손실은 (7)에 따라 $\mathbf{z}0$에서 마스킹된 토큰에 해당하는 로짓에 대해서만 계산됩니다. 생성 중에는 이 문제가 자연스럽게 해결되며, 연결이 필요하지 않습니다.

4.1 Eso-LM (A)

우리는 확산 단계 동안 노이즈 제거 트랜스포머에서 마스크 토큰에 대한 양방향 어텐션을 제거한 Eso-LM (A)를 소개합니다. 이러한 수정은 섹션 3.2에 설명된 샘플링 스케줄 사용을 가능하게 하여 더 빠른 추론을 허용합니다. Eso-LM (A)는 순차 샘플링 중에는 KV 캐싱을 지원하지만, 확산 샘플링 중에는 표준 MDM보다 훨씬 적은 계산을 수행합니다.

4.1.1 훈련

확산 단계 노이즈 제거 트랜스포머는 노이즈 제거할 마스크 토큰을 포함하는 $\mathbf{z}t \sim q_t(. | \mathbf{x})$을 입력으로, 그리고 $\mathbf{x}$을 타겟으로 받습니다. σ에서 $\mathbf{z}t$의 클린 토큰이 $\mathbf{z}t$의 마스크 토큰보다 먼저 와야 한다는 자연스러운 제약 조건을 가진 랜덤 순서 $\sigma \sim \mathcal{P}_L$가 샘플링됩니다. 우리는 L×L 어텐션 바이어스를 다음과 같이 정의합니다.

$$A_{i,j} = \begin{cases} 0 & \forall (i,j) \in C(z_t) \times C(z_t) \\ & \text{if } \sigma^{-1}(i) \geq \sigma^{-1}(j) \; \forall (i,j) \in M(z_t) \times [L] \\ -\infty & \text{otherwise.} \end{cases} \tag{10}$$

Clean tokens $\{\mathbf{z}t_i | i \in \mathcal{C}(\mathbf{z}t)\}$ have bidirectional attention among them (8), while a mask token $(\mathbf{z}t)_i \in \mathcal{M}(\mathbf{z}t)$ attends to clean tokens, itself and prior mask tokens per σ (9). Note that we ignore the

---

σ 내 클린 토큰 간의 순서를 무시하고 단순히 그들 사이의 양방향 어텐션을 사용합니다. 예시는 그림 2를 참조하십시오. 흥미롭게도, A의 행과 열을 σ에 따라 정렬하면 (그림 7), A은 Prefix-LM

순차 단계 노이즈 제거 트랜스포머는 마스크 토큰을 노이즈 제거할 $\mathbf{z}0 \sim q_0(. | \mathbf{x})$을 포함하는 연결된 시퀀스 $\mathbf{z}0 \oplus \mathbf{x} \in \mathcal{V}^{2L}$을 입력으로, 그리고 $\mathbf{x}$을 타겟으로 받습니다. 우리는 2L×2L 어텐션 바이어스를 다음과 같이 정의합니다.

$A_{i,j} = 0$	if $i = j \; \forall (i,j) \in M(z_0) \times M(z_0)$	(11)
$A_{i,j+L} = 0$	$\forall (i,j) \in M(z_0) \times C(z_0)$	(12)
$A_{i,j+L} = 0$	if $i > j \; \forall (i,j) \in M(z_0) \times M(z_0)$	(13)
$A_{i+L,j+L} = 0$	$\forall (i,j) \in C(z_0) \times C(z_0)$	(14)
$A_{i+L,j+L} = 0$	$\forall (i,j) \in M(z_0) \times C(z_0)$	(15)
$A_{i+L,j+L} = 0$	if $i \geq j \; \forall (i,j) \in M(z_0) \times M(z_0)$	(16)
$A_{i,j} = -\infty$	otherwise.	(17)

노이즈가 있는 문맥 $\mathbf{z}0$ 내 토큰에 대한 어텐션 메커니즘은 다음과 같습니다. 마스크 토큰 $(\mathbf{z}0)_i \in \mathcal{M}(\mathbf{z}0)$은 (i) 자기 자신 (11), (ii) 클린 토큰 $\{\mathbf{x}_j | j \in \mathcal{C}(\mathbf{z}0)\}$ (12), 그리고 (iii) 자신의 왼쪽에 있는 마스크 토큰의 클린 버전 $\{\mathbf{x}_j | j \in \mathcal{M}(\mathbf{z}0), i > j\}$ (13)에 어텐션합니다. 클린 토큰 $(\mathbf{z}0)_i \in \mathcal{C}(\mathbf{z}0)$는 다른 어떤 토큰도 그들에게 어텐션하지 않기 때문에 (17) 모든 것에 어텐션할 수 있습니다. 클린 문맥 $\mathbf{x}0$ 내 토큰에 대한 어텐션 메커니즘은 다음과 같습니다. 토큰 $\{\mathbf{x}_i | i \in \mathcal{C}(\mathbf{z}0)\}$은 양방향 어텐션을 가집니다 (14). 마스크 토큰에 해당하는 클린 토큰 $(\mathbf{x}_i) \in \mathcal{M}(\mathbf{z}0)$은 $\{\mathbf{x}_j | j \in \mathcal{C}(\mathbf{z}0)\}$ (15)과 $\{\mathbf{x}_j | j \in \mathcal{M}(\mathbf{z}0), i \geq j\}$ (16)에 어텐션합니다.

예시는 그림 6을 참조하십시오. σ를 두 가지 속성을 가진 순서라고 합시다: (i) σ에서 $\mathbf{z}0$의 클린 토큰이 $\mathbf{z}0$의 마스크 토큰보다 먼저 오고 (ii) σ에서 $\mathbf{z}0$의 마스크 토큰이 자연적인 순서로 되어 있습니다. 다시 말해, 클린 토큰 $\{\mathbf{x}_i | i \in \mathcal{C}(\mathbf{z}0)\}$ 간의 순서는 무시되며 우리는 그들 사이의 양방향 어텐션을 사용합니다. 각 4개의 L-by-L 블록의 행과 열이 σ에 따라 정렬될 때, A은 희소하고 패턴화된 블록 구조(그림 8)를 나타내며, 우리는 이를 구현에 활용합니다 (보충 자료 B.5).

4.1.2 샘플링

확산 또는 순차 샘플링 중에, 부분적으로 마스킹된 시퀀스 $\mathbf{z}^k$가 주어졌을 때, 노이즈 제거 모델은 $K=|\mathcal{S}|$인 $S^k \in \mathcal{S}=\{S^1,\cdots,S^K\}$에 대한 마스크 토큰 $\{\mathbf{z}^k_i | i \in S^k\}$을 노이즈 제거해야 합니다. 우리는 토큰들의 부분집합 $\{\mathbf{z}^k_i | i \in \mathcal{C}(\mathbf{z}^k) \cup S^k\}$에 대해 순방향 패스를 수행합니다. 토큰들의 부분집합에 대해 순방향 패스를 수행하는 동안, 실제 시퀀스에서 이 토큰들의 위치 임베딩이 보존된다는 점에 유의하는 것이 중요합니다. 아래에서는 순방향 패스에 사용된 어텐션 바이어스에 대해 논의합니다.

k 단계 이전에 확산 단계에서 디코딩된 토큰들의 인덱스 집합을 D^k_{MDM}이라고 하고, 순차 단계의 인덱스 집합을 D^k_{AR}라고 합시다. S^k에 의해 정의된 토큰들을 노이즈 제거하는 순서를 σ이라고 합시다. 우리는 k 단계에서 $L \times L$ 어텐션 바이어스를 다음과 같이 정의합니다.

$A_{i,j} =$	0	$\forall (i,j) \in D^k_{MDM} \times D^k_{MDM}$	(18)
	0	$\forall (i,j) \in D^k_{AR} \times D^k_{MDM}$	(19)
	0	if $i \geq j \; \forall (i,j) \in D^k_{AR} \times D^k_{AR}$	(20)
	0	$\forall (i,j) \in S^k \times (D^k_{MDM} \cup D^k_{AR})$	(21)
	0	if $\sigma^{-1}(i) \geq \sigma^{-1}(j) \; \forall (i,j) \in S^k \times S^k$	(22)
	$-\infty$	otherwise.	(23)

확산 단계에서 디코딩된 클린 토큰 $\{\mathbf{z}^k_i | i \in D^k_{MDM}\}$은 그들 사이에서 양방향 어텐션을 가집니다 (18). 순차적으로 디코딩된 클린 토큰 $(\mathbf{z}^k_i)_{i \in D^k_{AR}}$은 확산 단계에서 디코딩된 클린 토큰 $\{\mathbf{z}^k_j | j \in D^k_{MDM}\}$ (19), 자기 자신, 그리고 순차적으로 디코딩된 이전 클린 토큰 $\{\mathbf{z}^k_j | j \in D^k_{AR}, i > j\}$ (20)에 어텐션합니다. 노이즈 제거할 마스크 토큰 $(\mathbf{z}^k_i)_{i \in S^k}$는 모든 디코딩된 클린 토큰 $\{\mathbf{z}^k_j | j \in D^k_{MDM} \cup D^k_{AR}\}$ (21), 자기 자신, 그리고 σ에 따라 노이즈 제거할 이전 마스크 토큰: $\{\mathbf{z}^k_j | j \in S^k, \sigma^{-1}(i) > \sigma^{-1}(j)\}$ (22)에 어텐션합니다. 노이즈 제거할 예정이 아닌 마스크 토큰 $(\mathbf{z}^k_i)_{i \in S^{>k}}$은 다른 어떤 토큰도 그들에게 어텐션하지 않기 때문에 (23) 모든 것에 어텐션할 수 있습니다.

(i) $(\mathbf{z}^k_i)_{i \in \mathcal{C}(\mathbf{z}^k) \cup S^k}$이 $\{\mathbf{z}^k_i | i \in S^{>k}\}$에 어텐션하지 않고 (ii) 클린 토큰 샘플링을 위해 $\{\mathbf{z}^k_i | i \in S^k\}$에 대한 트랜스포머 출력만 사용하므로, $\{\mathbf{z}^k_i | i \in S^{>k}\}$에 대한 어떤 계산도 피할 수 있어 확산 단계 동안 계산량이 현저히 줄어듭니다. 그림 9는 이 샘플링 절차가 순차 단계 동안 KV 캐싱을 사용하여 예시 시퀀스를 생성하는 방법을 보여줍니다.

4.2 Eso-LM (B)

Eso-LM (A)는 순차 샘플링 중에 KV 캐싱을 수행할 수 있지만, 클린 토큰이 그들 사이에서 양방향 어텐션을 가지기 때문에 확산 샘플링 중에는 이를 수행할 수 없습니다. 따라서, 한 세트의 마스크 토큰이 노이즈 제거된 후, 이제 새로 노이즈 제거된 토큰을 포함한 모든 클린 토큰에 대한 KV가 업데이트되어야 합니다. 확산 샘플링 중에 KV 캐싱을 가능하게 하기 위해, 우리는 클린 토큰들 간에 인과적 어텐션을 강제하며, 이는 Eso-LM (B)로 이어집니다. 또한 우리는 두 단계 간에 공유 KV 캐시를 가능하게 합니다.

4.2.1 훈련

확산 단계 노이즈 제거 트랜스포머는 노이즈 제거할 마스크 토큰을 포함하는 $\mathbf{z}_t \sim q_t(. | \mathbf{x})$을 입력으로, 그리고 $\mathbf{x}$을 타겟으로 받습니다. 우리는 MDM과 AO-ARM

$$A_{i,j} = \{0 &\text{if } \sigma^{-1}(i) \geq \sigma^{-1}(j) \;\; \forall (i,j) \in [L] \times [L] \\ -\infty &\text{otherwise.} \\ \tag{24}$$

예시는 그림 2를 참조하십시오. 또한, A의 행과 열을 σ에 따라 정렬하면 (그림 7), A은 인과적 어텐션 바이어스가 됩니다. 우리는 이 속성을 구현에 활용합니다 (보충 자료 B.5).

순차 단계 노이즈 제거 트랜스포머는 마스크 토큰을 노이즈 제거할 $\mathbf{z}0 \sim q0(.|\mathbf{x})$을 포함하는 연결된 시퀀스 $\mathbf{z}0 \oplus \mathbf{x} \in \mathcal{V}2L$을 입력으로, 그리고 $\mathbf{x}$을 타겟으로 받습니다. 샘플링 중에는, 인과적 어텐션이 적용되고 KV 캐시가 구축되는 $\mathbf{z}0 \sim p\phi$MDM 내 클린 문맥은 확산 단계에 의해 제공됩니다. 확산 단계에서 구축된 KV 캐시를 효과적으로 사용하기 위해, 우리는 순차 단계 훈련 동안 클린 토큰 $\{\mathbf{x}i|i \in \mathcal{C}(\mathbf{z}0)\}$ 간에 유사한 인과적 어텐션을 강제해야 합니다. 이를 달성하기 위해, 우리는 먼저 (i) σ에서 $\mathbf{z}0$의 클린 토큰이 $\mathbf{z}0$의 마스크 토큰보다 먼저 오고 (ii) σ에서 마스크 토큰이 자연적인 순서로 되어 있다는 제약 조건을 가진 $\sigma \sim \mathcal{P}L$를 샘플링합니다. 그 다음, 우리는 (11)-(17)을 재사용하지만, $\{\mathbf{x}i|i \in \mathcal{C}(\mathbf{z}0)\}$ (14)에 대한 양방향 어텐션을 σ (29)에 따른 인과적 어텐션으로 대체합니다.

$$Ai,j=0 \text{if } i=j \ \forall (i,j) \in \mathcal{M}(\mathbf{z}0) \times \mathcal{M}(\mathbf{z}0) ₩ \text{tag26}$$

$$Ai,j+L=0 \forall (i,j) \in \mathcal{M}(\mathbf{z}0) \times \mathcal{C}(\mathbf{z}0) ₩ \text{tag27}$$

$$Ai,j+L=0 \text{if } i>j \ \forall (i,j) \in \mathcal{M}(\mathbf{z}0) \times \mathcal{M}(\mathbf{z}0) ₩ \text{tag28}$$

$$Ai+L,j+L=0 \text{if } \sigma-1(i) \geq \sigma-1(j) \ \forall (i,j) \in \mathcal{C}(\mathbf{z}0) \times \mathcal{C}(\mathbf{z}0) ₩ \text{tag29}$$

$$Ai+L,j+L=0 \forall (i,j) \in \mathcal{M}(\mathbf{z}0) \times \mathcal{C}(\mathbf{z}0) ₩ \text{tag30}$$

$$Ai+L,j+L=0 \text{if } i \geq j \ \forall (i,j) \in \mathcal{M}(\mathbf{z}0) \times \mathcal{M}(\mathbf{z}0) ₩ \text{tag31}$$

$$Ai,j=-\infty \text{otherwise.} ₩ \text{tag32}$$

예시는 그림 6을 참조하시오. 네 개의 L×L 블록 각각의 행과 열이 σ로 정렬될 때, A는 구현에서 활용하는 구조(그림 8)를 표시한다(보충 자료 B.5).

4.2.2 샘플링

섹션 4.1.2의 동일한 설정 및 표기법에 따라, 우리는 (18)-(23)을 재사용하지만, DkMDM (18)에 대한 양방향 어텐션을 σ당 인과적 어텐션, 즉 노이즈 제거 순서 (33)으로 대체한다:

$$Ai,j=\{0amp;\text{if } \sigma-1(i) \geq \sigma-1(j) \ \forall (i,j) \in DkMDM \times DkMDM 0amp; \forall (i,j) \in DkAR \times DkMDM 0amp; \text{if } i \geq j \ \forall (i,j) \in DkAR \times DkAR 0am$$
₩tag38

두 단계에서의 KV 캐싱 섹션 4.1.2에서와 같이, 여기서는 미래 마스크 토큰 $\{\mathbf{z}ki|i \in S>k\}$에 대한 계산을 수행하지 않는다. 또한, 이전에 디코딩된 토큰(확산에 의해 디코딩되었든 순차적으로 디코딩되었든 관계없이)은 서로 인과적 어텐션을 가지므로, 스텝 k에서 새로 생성된 토큰은 스텝 k+1에서 KV 캐시를 구축하는 데 사용될 수 있으며, 스텝 k+2부터 캐시된 KV 값이 사용될 수 있다. 그림 1은 이 샘플링 절차가 예시 시퀀스를 어떻게 생성하는지 보여준다.

---

5 실험

우리는 두 가지 표준 언어 모델링 벤치마크, 즉 One Billion Words 데이터셋 (LM1B) *

5.1 우도 평가

우리의 실험은 Eso-LM이 LM1B와 OWT에서 확산 모델 중 최첨단 혼란도를 달성하며, MDM과 AR 혼란도 사이의 더 미세한 보간을 제공함을 보여준다(표 1 및 표 2).

실험 설정 우리는 선도적인 마스크 확산 모델—MDLM *

결과 우리 방법의 경우, PPL은 (7)을 사용하여 계산된다. LM1B에서 우리는 $\alpha_0 \in \{0.0625, 0.125, 0.25, 0.5, 1.0\}$으로 Eso-LM (A)를 훈련하고 $\alpha_0 = 1$으로 Eso-LM (B)를 훈련한다. 우리는 Eso-LM (A)가 MDLM과 AR 혼란도 사이를 효과적으로 보간한다는 것을 발견한다. 놀랍게도, 마스크 토큰에 대한 양방향 어텐션을 인과적 마스크로 대체하여 MDM을 수정하는 Eso-LM (A)는 $\alpha_0 = 1$에서 MDLM보다 ≈ 1 PPL 더 우수하다. 이러한 개선은 앞에서 설명한 저분산 훈련 손실에 기인할 수 있다. 대조적으로, Eso-LM (B)는 깨끗한 토큰들 사이에 인과적 종속성을 도입하여 $\alpha_0 = 1$에서 Eso-LM (A)에 비해 성능을 ≈ 4 PPL 저하시킨다. 이는 Eso-LM (A)의 깨끗한 토큰에 대한 양방향 어텐션이 Eso-LM (B)의 인과적 어텐션보다 엄밀히 더 표현력이 높기 때문에 예상되는 결과이다. OWT에서 Eso-LM (B)는 확산 단계에서 KV 캐싱을 지원하는 유일한 모델(BD3-LMs 포함)이라는 점에서 특히 설득력 있다. $\alpha_0 \in \{0.0625, 0.125, 0.25, 0.5, 1.0\}$으로 훈련했을 때, AR과 MDM 혼란도 사이의 부드러운 보간을 보여준다(표 2). LM1B와 마찬가지로, Eso-LM (A)는 OWT에서 MDLM과 경쟁력을 유지한다.

5.2 생성 속도

우리의 정성적 평가 실험은 Eso-LM이 KV 캐싱을 지원하는 이전 확산 기반 방법보다 3–4× 더 빠르며, KV 캐싱을 지원하지 않는 MDM보다 최대 65× 더 빠르다는 것을 보여준다.

실험 설정 MDLM과 같은 확산 모델은 양방향 어텐션 사용으로 인해 KV 캐싱을 지원하지 않는다. BD3-LM과 같은 준-자기회귀 모델은 토큰 블록을 자기회귀적으로 생성하고 완료된 블록을 캐싱함으로써 부분적인 KV 캐싱을 가능하게 하지만, 확산 중인 현재 블록에 대한 캐싱은 여전히 부족하다. 대조적으로, 우리 방법은 확산 단계에서 KV 캐싱을 고유하게 지원한다. 우리는 컨텍스트 길이 $L \in \{2048, 8192\}$에서 MDLM 및 BD3-LM과 우리 방법인 Eso-LM (B)의 추론 시간을 비교하며, 첫 방문 샘플러 *

표 1: LM1B의 테스트 혼란도 (PPL; ↓). *He et al.

	PPL (↓)
Autoregressive	
Transformer‡	22.83
Diffusion	
D3PM Uniform [2]	137.90$
D3PM Absorb [2]	76.90$
Diffusion-LM* [12]	118.62$
DiffusionBert [7]	63.78
SEDD Absorb‡ [15]	32.71$
SEDD Uniform$ [15]	40.25$
MDLM‡ [26]	31.78
UDLM‡ [29]	36.71
DUO‡ [27]	33.68
Interpolating diffusion and autoregressive	
BD3-LMs† [1]	
L' = 16	30.60
L' = 8	29.83
L' = 4	28.23

	PPL (↓)
Eso-LM (B) (Ours)	
α0 = 1.0	35.00
Eso-LM (A) (Ours)	
α0 = 1.0	30.96
α0 = 0.5	30.51
α0 = 0.25	28.44
α0 = 0.125	25.97
α0 = 0.0625	24.51

표 2: 25만 스텝 훈련 모델에 대한 OWT의 테스트 혼란도 (PPL; ↓). ǂ

	PPL (↓)
Autoregressive	
Transformer	17.90$
Diffusion	
SEDD Absorb [15]	26.81$
MDLM [26]	25.76$
UDLM [29]	30.52$
DUO [27]	27.14$
Interpolating diffusion and autoregressive	
BD3-LMs [1]	
L' = 16	23.57†
Eso-LM (A) (Ours)	
α0 = 1	26.21
Eso-LM (B) (Ours)	
α0 = 1	30.14
α0 = 0.5	27.85
α0 = 0.25	24.73
α0 = 0.125	21.87

표 3: 모든 방법에 대해 NFEs를 L로 설정했을 때 시퀀스 길이 L∈{2048,8192}에 대한 샘플링 시간 (↓) (초). 보고된 값은 5회 이상의 실행에 대한 평균이다.

Method	L = 2048	L = 8192
AR (Oracle)	13.30.9	54.00.2
MDLM	201.30.4	5438.33.3
BD3-LM (L' = 4)	24.30.7	312.01.7

Method	L = 2048	L = 8192
BD3-LM (L' = 16)	21.30.1	268.11.2
Eso-LM (B)	14.60.3	82.10.3

결과 표 3에 나타난 바와 같이, KV 캐싱이 없는 MDLM과 비교할 때, Eso-LM (B)는

L=2048에서는 ~14×배 빠르며, L=8192에서는 ~65×배 빠르다. 부분적으로 캐싱을 지원하는 BD3-LM과 비교했을 때, Eso-LM (B)는 L=8192에서 BD3-LM (L′=16)보다 ~3.2×배 빠르며 BD3-LM (L′=4)보다 ~3.8×배 빠르다. 이러한 속도 향상은 KV 캐싱과, 디노이징되어야 할 마스크 토큰과 이전에 예측된 토큰으로 순방향 패스를 제한하여 중복 계산을 피하는 스케줄러 S에서 비롯된다. 이는 BD3-LM에는 없는 기능이다. NFEs를 L으로 제한하면, 우리의 방법은 지연된 KV 재사용(마지막에서 두 번째 디노이징 스텝부터만 가능)으로 인해 AR 모델보다 느리지만, 전반적인 효율성은 이전 확산 기반 베이스라인을 여전히 능가한다(섹션 4.2.2 참조).

5.3 생성 품질

우리의 샘플링 실험은 (1) Eso-LM이 샘플링 속도와 품질의 파레토 프론티어에서 새로운 SOTA를 확립하고(그림 4), (2) 높은 NFE에서 모든 이전 확산 모델보다 고품질 샘플을 생성하며, (3) 이전 보간 확산 방법인 BD3-LM과는 달리 낮은 NFE에서 모드 붕괴를 겪지 않음을 보여준다(그림 3).

실험 설정 우리는 α0train∈{0.125,0.25,1}으로 Eso-LM (B)를 훈련하고, NFE를 제어하기 위해 (NFEs = |S|) 다양한 (α0eval,T)∈{0.0625,0.25,0.5,1}×{16,128,1024}을 사용하여 샘플을 생성한다. MDLM과 BD3-LM은 Sahoo et al.

!

!

1 미만이다. 우리는 샘플 품질을 위해 생성 혼란도(GPT-2 Large를 통해)를 측정하고, 다양성을 위해 평균 엔트로피

생성 PPL 대 NFE의 파레토 프론티어 그림 3에서 볼 수 있듯이, 높은 NFE (≈ 1024)에서 Eso-LM (B)는 MDLM과 BD3-LM을 능가하며 AR 모델 품질에 거의 근접한다. NFE가 감소함에 따라, BD3-LM은 교사 강제 훈련으로 인해 성능이 크게 저하된다. 즉, 훈련 중에는 미래 블록이 깨끗한 이전 블록에 조건화되지만, 평가 중에는 이전 블록 예측의 오류가 샘플 품질의 상당한 저하로 이어진다. Eso-LM은 이 문제를 피한다. 예상대로, 낮은 NFE 영역에서는 더 높은 α0train(훈련 중 "더 많은 확산")가 품질을 향상시킨다: α0train=1는 α0train={0.125,0.25,1} 중에서 가장 좋은 성능을 보이며, MDLM과 경쟁력을 유지한다. 전반적으로, Eso-LM (B)는 낮은 NFE에서 MDLM과 경쟁하고 높은 NFE에서 AR 모델과 경쟁한다—BD3-LM과는 다르다.

생성 PPL 대 속도의 파레토 프론티어 샘플링 효율성을 비교하기 위해, 우리는 각 방법이 단일 샘플을 생성하는 데 걸리는 샘플링 시간의 중앙값(5회 시험에 걸쳐)을 초 단위로 기록하고, 그림 4에 생성 PPL 대 샘플링 시간을 플롯한다. 무한 시험의 한계에서, 샘플링 시간의 중앙값은 NFE의 증가 함수이며, 사용된 방법과 샘플링 하이퍼파라미터에 의해 조절된다. 각 방법에 대해, 우리는 모든 구성에 걸쳐 파레토 프론티어를 구축한다: Eso-LM (B) (α0train, α0eval, T에 걸쳐), BD3-LM (L′ 및 T에 걸쳐), 그리고 MDLM (T에 걸쳐). 우리는 Eso-LM (B)가 속도-품질 파레토 프론티어에서 새로운 최첨단 기술을 확립한다는 것을 발견한다. 샘플링 시간 예산이 낮은 경우, α0train=1으로 훈련된 모델이 가장 좋으며; 예산이 높은 경우, α0train=0.125 모델은 AR에 가까운 품질을 생성한다. 중간 값(예: α0train={0.25,0.5})은 속도와 품질 사이의 최상의 절충안을 제공한다.

6 관련 연구, 논의 및 결론

AR 모델은 토큰을 왼쪽에서 오른쪽으로 생성하며 품질 면에서 최신 기술(state-of-the-art)을 유지하지만, 느리고 순차적인 추론과 제한된 제어 가능성이라는 단점을 가지고 있습니다. 대조적으로, Eso-LM은 순차적 단계에서 AR과 유사한 생성과 초기 확산 단계에서 임의 순서의 병렬 생성을 결합합니다. 확산 과정 동안 Eso-LM (B)는 이전에 AR 모델에만 국한되었던 KV 캐싱

마스크 확산 MDM [26, 30]은 단계별로 여러 토큰을 생성하지만, 비용이 많이 드는 양방향 어텐션에 의존합니다. Eso-LM은 두 가지 방식으로 효율성을 개선합니다. Eso-LM (A)는 어텐션을 깨끗한 토큰과 노이즈 제거 예정 토큰으로 제한하며, Eso-LM (B)는 AO-ARM

블록 확산 BD3-LM

훈련 중 교사 강요(teacher forcing) 사용으로 인해 후속 블록의 샘플 품질에 심각한 영향을 미칩니다. Eso-LM은 이러한 문제로 고통받지 않습니다.

결론

우리는 자기회귀(AR) 모델과 마스크 확산 모델(MDM)을 원활하게 융합하여 샘플 품질과 생성 속도 모두에서 두 모델 간의 효과적인 보간을 가능하게 하는 언어 모델링을 위한 새로운 패러다임을 소개합니다. 우리 방법은 병렬 생성을 유지하면서 MDM에서 KV 캐싱을 가능하게 한 최초의 방법이며, 추론을 크게 가속화합니다. 이는 속도와 정확도 면에서 블록 확산 방법을 능가하며, 언어 모델링 벤치마크에서 새로운 최신 기술(state of the art)을 확립합니다. 언어 모델링 연구를 진행함에 따라, 이 분야에 내재된 위험과 기회가 따릅니다.

참고문헌

-

---

[14] Chengyi Liu, Wenqi Fan, Yunqing Liu, Jiatong Li, Hang Li, Hui Liu, Jiliang Tang, and Qing Li. Generative diffusion models on graphs: Methods and applications. arXiv preprint arXiv:2302.02591, 2023.

- [15] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. arXiv preprint arXiv:2310.16834, 2024.
- [16] Mitch Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. Computational linguistics, 19(2):313-330, 1993.
- [17] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
- [18] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. arXiv preprint arXiv:2502.09992, 2025.
- [19] Hunter Nisonoff, Junhao Xiong, Stephan Allenspach, and Jennifer Listgarten. Unlocking guidance for discrete state-space diffusion and flow models. arXiv preprint arXiv:2406.01572, 2024.
- [20] Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=sMyXP8Tanm.
- [21] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernandez. The LAMBADA dataset: Word

prediction requiring a broad discourse context. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1525-1534, Berlin, Germany, August 2016. Association for Computational Linguistics. URL http://www.aclweb.org/anthology/P16-1144.

- [22] William Peebles and Saining Xie. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 4195-4205, 2023.
- [23] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference, 2022. URL https://arxiv.org/abs/2211.05102.
- [24] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [25] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of machine learning research, 21(140):1-67, 2020.
- [26] Subham Sekhar Sahoo, Marianne Arriola, Aaron Gokaslan, Edgar Mariano Marroquin, Alexander M Rush, Yair Schiff, Justin T Chiu, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024. URL https://openreview.net/forum?id=L4uaAR4ArM.
- [27] Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, Guanghan Wang, Justin T Chiu, and Volodymyr Kuleshov. The diffusion duality. In ICLR 2025 Workshop on Deep Generative Model in Machine Learning: Theory, Principle and Efficacy, 2025. URL https://openreview.net/forum?id=CB0Ub2yXjC.
- [28] Yair Schiff, Chia-Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. Caduceus: Bi-directional equivariant long-range dna sequence modeling. arXiv preprint arXiv:2403.03234, 2024.

---

[29] Yair Schiff, Subham Sekhar Sahoo, Hao Phung, Guanghan Wang, Sam Boshar, Hugo Dallatorre, Bernardo P de Almeida, Alexander M Rush, Thomas PIERROT, and Volodymyr Kuleshov. Simple guidance mechanisms for discrete diffusion models. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=i5MrJ6g5G1.

- [30] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data, 2025. URL https://arxiv.org/abs/2406.04329.
- [31] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In International conference on machine learning, pp. 2256-2265. PMLR, 2015.
- [32] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. Advances in neural information processing systems, 32, 2019.
- [33] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. arXiv preprint arXiv:2104.09864, 2021.
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.
- [35] Guanghan Wang, Yair Schiff, Subham Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling. arXiv preprint arXiv:2503.00307, 2025.
- [36] Xiang Zhang, Junbo Jake Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In NIPS, 2015.

- [37] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. arXiv preprint arXiv:2409.02908, 2024.

목차

1 서론 1 2 배경 2 2.1 자기회귀(Autoregressive) 모델 3 2.2 마스크 확산(Masked Diffusion) 모델 3 2.3 블록 노이즈 제거 확산 이산 언어 모델 3 3 에소테릭(Esoteric) 언어 모델 4 3.1 자기회귀 모델과 마스크 확산의 융합 4 3.2 샘플링 5 4 공유된 노이즈 제거 트랜스포머를 위한 어텐션 메커니즘 설계 5 4.1 Eso-LM (A) 6 4.2 Eso-LM (B) 8 5 실험 9 5.1 우도 평가 9 5.2 생성 속도 9 5.3 생성 품질 10 6 관련 연구, 논의 및 결론 11 부록 16 부록 A 배경 16 A.1 BD3-LM 하이퍼파라미터 T′ 및 num_tries 16 부록 B 에소테릭 언어 모델 17 B.1 MDM 손실 17 B.2 훈련 절차 18 B.3 순차적 단계에 대한 어텐션 편향 18 B.4 샘플링 스케줄 및 샘플러 18 B.5 구현 세부 사항 19 B.6 실험 세부 사항 20 부록 C 추가 실험 22 C.1 분할 비율에 대한 제거 연구 22 C.2 OWT 혼란도 22 C.3 제로샷 우도 평가 22 C.4 OWT로 훈련된 모델에 의해 생성된 샘플의 생성 혼란도 및 엔트로피 23

부록

부록 A 배경

A.1 BD3-LM 하이퍼파라미터 T′ 및 num_tries

BD3-LM

또한, 우리는 BD3-LM의 코드베이스가 샘플을 선별적으로 선택한다는 것을 발견했다. 더 구체적으로 말하면, 단일 샘플을 생성하기 위해 코드베이스는 하나의 샘플이 어떤 품질 관리 테스트를 통과할 때까지 계속해서 새 샘플을 생성한다(num_tries번까지). 기본적으로 num_tries는 10으로 설정되며, 코드베이스는 10번의 시도가 모두 소진되었지만 테스트를 통과한 샘플이 없을 때 샘플링 실패를 보고한다. 경험적으로, 우리는 T′=5000의 경우 샘플링 실패가 발생하지 않는다는 것을 발견했다.

병렬 생성을 위한 BD3-LM의 실제 성능을 조사하기 위해, 우리는 num_tries를 1로 설정하고, 품질 관리 테스트를 비활성화했으며, 광범위한 T 값에 걸쳐 BD3-LM의 샘플을 평가했다(그림 5). 여기 및 그림 5에서 T은 BD3-LM의 모든 블록에 걸친 샘플링 단계의 합계를 의미한다. 예를 들어, L′=16 및 T=4096은 블록당 T′=4096/(1024/16)=64 샘플링 단계가 사용됨을 의미한다. 대조적으로, BD3-LM의 코드베이스는 기본적으로 T′=5000를 사용하는데, 이는 그림 5의 T=∞에 해당한다. MDLM의 경우, 블록이 없으므로 T은 정상적으로 해석될 수 있다.

그림 5에서 보듯이, 더 많은 병렬 생성을 가능하게 하기 위해 T이 감소함에 따라, BD3-LM의 샘플 품질과 샘플 다양성 모두 MDLM보다 현저히 나빠진다. 이러한 성능 저하는 섹션 6에서 논의된 바와 같이 모드 붕괴 문제 때문이다. 우리는 또한 num_tries를 늘리면 BD3-LM의 샘플 엔트로피를 어느 정도 향상시킬 수 있고(표 4의 두 번째 행) 모드 붕괴를 완화할 수 있지만, 그렇게 해도 AR 및 MDLM에는 개선 효과가 적거나 전혀 없다는 것을 발견했다.

!

표 4: 핵 샘플링 p=0.9을 사용한 BD3-LM L′=16, MDLM 및 AR의 생성 PPL (↓) 및 엔트로피 (↑) (괄호 안). 우리는 BD3-LM을 위해

	BD3LM		MDLM		AR	
num_tries	1	10	1	10	1	10
T=1024	72.80 (5.35)	77.71 (5.41)	41.92 (5.36)	41.79 (5.37)	13.03 (5.26)	13.76 (5.32)
T=256	356.02 (5.11)	440.69 (5.28)	45.07 (5.40)	44.57 (5.39)	13.03 (5.26)	13.76 (5.32)

부록 B 난해한 언어 모델

B.1 MDM 손실

NLL은 다음과 같이 주어진다:

−logpθ(x)≤−Ez0~q0(·|x)logpθAR(x|z0)+DKL(q0(z0|x) ‖ pθMDM(z0))=−Ez0~q0(·|x) [Σℓ∈M(z0)logpθAR(xℓ|z0,x<ℓ)]+DKL(q0(z0|x) ‖ pθMDM(z0)).₩tag39

z0은 인덱스 ℓ을 초과하는 인덱스에 깨끗한 토큰을 포함할 수 있음에 유의하라. 섹션 3.1에서 논의된 바와 같이, AR 로그-우도는 다음과 같이 주어진다:

Σℓ∈M(z0)logpθAR(xℓ|z0,x<ℓ)=Σℓ∈M(z0)log(xθℓ(z0⊙x<ℓ),xℓ),₩tag40

여기서 우리는 마스크된 인덱스 $\mathscr{M}(\mathbf{z}0)$에서만 손실을 계산한다.

(39)의 KL 항을 계산하기 위해, 우리는 **z**0에 대한 마스크된 확산 프로세스를 정의한다. 이 확산 프로세스의 경우, 시간 t∈[0,1]에서의 순방향 마진 **z**tℓ~qt(·|**x**ℓ)은 (1)과 동일하지만, 축소된 범위 (αt)t∈[0,1]∈[0,α0]을 갖는 노이즈 스케줄을 사용한다. 이는 t에서 엄격하게 감소하는 함수이며, αt=0=α0가 **z**t=0=**z**0를 만족하고 αt=1=0가 **z**t=1=**m**1:L를 만족하도록 한다. T 확산 단계를 통해, 우리는 다음을 얻는다:

DKL(q0(z0|x) ‖ pθMDM(z0))=Ez0[logq0(z0|x)pθMDM(z0)]=Ez0[logEz1T:1[q(z0:1|x)pθMDM(z0:1)]]≤Ez0:1[logq(z0:1|x)pθMDM(z0:1)]=Ez0:1[Σt∈{1T,2T,⋯,1}logq(zt−1T|zt,x)pθMDM(zt−1T|zt)]=ΣtEzt[DKL(q(zt−1T|zt,x) ‖ pθMDM(zt−1T

Sahoo 등

=Et~U[0,1],zt~qt[αt′1−αtΣℓ∈M(zt)log(xθℓ(zt),xℓ)].₩tag41

마지막으로, (40)과 (41)을 결합하면 원하는 결과를 얻는다:

LNELBO(x;θ)=Ez0~q0[−Σℓ∈M(z0)log(xθℓ(z0⊙x<ℓ),xℓ)⏟ARloss] +∫t=0t=1αt′1−αtEzt~qt[Σℓ∈M(zt)log(xθℓ(zt),xℓ)]dt⏟MDMloss.₩tag42

B.2 훈련 절차

알고리즘 1은 완전한 훈련 절차를 요약한다.

알고리즘 1 Eso-LM 훈련 입력: 데이터셋 D, 배치 크기 bs, 순방향 노이즈 프로세스 qt(·|**x**), 모델 **x**θ, 학습률 η 수렴할 때까지 반복 **x**1,**x**2,⋯,**x**bs~D i←1부터 bs/2까지 반복 ▷ 만약 α0=1이면, 1부터 bs까지 반복한다. **z**0~q0(·|**x**) 제약 조건 σ~$\mathcal{P}$L (섹션 4의 **x**θ에서 어텐션 편향 A을 구성하는 데 사용됨) (7)의 순차 손실 추정기 $\mathscr{A}$←−Σℓ∈$\mathscr{M}$(**z**0)log(**x**θℓ(**z**0,**x**<ℓ),**x**iℓ) 반복 종료 i←bs/2+1부터 bs까지 반복 ▷ 만약 α0=1이면, 이 반복을 건너뛴다. t~$\mathcal{U}$[0,1] 샘플링 **z**t~qt(·|**x**) 제약 조건 σ~$\mathcal{P}$L (섹션 4의 **x**θ에서 어텐션 편향 A을 구성하는 데 사용됨) (7)의 MDM 손실 추정기 $\mathscr{A}$←αi′1−αiΣℓ∈$\mathscr{M}$(**z**t)log(**x**θℓ(**z**t),**x**iℓ) 반복 종료 θ←θ−η▽θΣi=1bs$\mathscr{A}$ 반복 종료

B.3 순차 단계의 어텐션 편향

그림 6은 순차 단계 훈련을 위한 어텐션 편향의 비교를 보여준다.

!

B.4 샘플링 스케줄 및 샘플러

Eso-LM은 두 가지 샘플링 단계를 수행한다: 확산 단계와 순차 단계. 확산 단계 내에서 토큰은 무작위 순서로 그리고 잠재적으로 병렬로 디노이징된다. 순차 단계 내에서 남아있는 마스크 토큰은 왼쪽에서 오른쪽으로 그리고 한 번에 하나씩 순차적으로 디노이징된다.

먼저, (i) 확산 단계 동안 디노이징할 총 토큰 수와 (ii) 확산 단계당 디노이징할 토큰 수를 결정하기 위해, 우리는

T 단계. 각 단계마다, 우리는 시간 t에 디노이징할 토큰 수를 다음과 같이 계산한다:

nt=Binom(n=ntremaining,p=αs−αt1−αt), (43)

여기서 s=t−dt 및 ntremaining=L−Σt′>tnt′이다. T가 클 때, 일부 nt은 0일 수 있다. 이 알고리즘에 의해 생성된 모든 nt는 0인 nt를 제외하고 정렬된 목록에 수집된다. 우리는 모든 nt의 합계를 nMDM으로 나타내고 nAR=L−nMDM로 정의한다.

우리는 [L]에서 nMDM개의 토큰 인덱스를 선택하여 확산 방식으로 디노이징하고, 나머지 보완적인 토큰 인덱스 부분집합은 순차적으로 디노이징한다. 예를 들어, L=8이고 토큰 인덱스가 [1,2,⋯,8]이라고 가정하자. 위 알고리즘에서 nMDM=5를 얻었다고 가정하자. 그러면 우리가 선택할 확산 인덱스는 (1,3,4,6,7)이고, 보완적인 순차 인덱스는 (2,5,8)이다. 우리는 무작위 순서 디노이징을 위해 확산 인덱스를 예를 들어 (3,1,6,4,7)와 같이 무작위로 재배열한다.

0이 아닌 nt의 목록과 확산 지수의 순열 정렬 집합이 주어졌을 때, nt에 따라 확산 지수를 분할하여 확산 샘플링 스케줄을 생성합니다. 0이 아닌 nt의 목록이 (2,1,2)이라고 가정합니다. 이를 사용하여 확산 지수 (3,1,6,4,7)의 순열 집합을 분할하면 확산 단계에 대한 다음 샘플링 스케줄을 얻습니다: $\mathcal{S}$MDM=((3,1),(6),(4,7)). 순차 단계에 대한 노이즈 제거 스케줄은 단순히 $\mathcal{S}$AR=((2),(5),(8))입니다. 통합 샘플링 스케줄 $\mathcal{S}$은 $\mathcal{S}$MDM과 $\mathcal{S}$AR의 연결입니다. 이 예에서 $\mathcal{S}$=(S1,S2,S3,S4,S5,S6) (여기서 S1=(3,1),S2=(6),S3=(4,7),S4=(2),S5=(5) 및 S6=(8))입니다. 이는 6개의 NFE에 해당합니다. 마지막으로, $\mathcal{S}$이 알고리즘 2로 전달되며, 알고리즘 2는 나머지 샘플링 절차를 처리합니다. 섹션 4.1.2 및 섹션 4.2.2에서 논의된 노이즈 제거 순서 σ와 연결하면, 이 예에서는 σ=(3,1,6,4,7,2,5,8)입니다.

입력: 시퀀스 길이 L, 통합 샘플링 스케줄 $\mathcal{S}$ $\mathbf{z}$=[MASK_INDEX,⋯,MASK_INDEX] C={} ▷ 클린 토큰의 인덱스 i←1부터 |$\mathcal{S}$|까지 반복 ▷ |C|≥nMDM일 때 순차적으로 자동 발생 logits ←$\mathbf{x}_\theta$($\mathbf{z}$[C∪Si]) ▷ 비고 참조 logits ← Si에 해당하는 logits 선택 $\mathbf{z}$[Si]←categorical_sample(logits, dim=-1) ▷ logits의 형태는 (|Si|,|$\mathcal{V}$|) C←C∪Si 반복 종료 반환: $\mathbf{z}$ 비고. $\mathbf{z}$[C∪Si]은 노이즈 제거 모델 $\mathbf{x}_\theta$에 공급되는 $\mathbf{z}$ 내 토큰의 부분 집합을 나타냅니다. 토큰 $\mathbf{z}^\ell \in \mathbf{z}$[C∪Si]에 대한 위치 임베딩은 원래 시퀀스 $\mathbf{z}$에서의 위치 임베딩과 동일하게 보장됩니다. Eso-LM (A) 및 Eso-LM (B)에 대한 샘플링 어텐션 편향 A 계산은 각각 섹션 4.1.2 및 섹션 4.2.2를 참조하십시오. Eso-LM (B)의 경우, 인과적 어텐션(causal attention)의 사용으로 인해 $\mathbf{x}_\theta$는 클린 토큰이 처음 처리될 때 KV-값을 캐싱할 수 있습니다.

알고리즘 2 Eso-LM 샘플링

B.5 구현 세부 사항

B.5.1 훈련

확산 단계

Eso-LM (A) 및 Eso-LM (B) 모두에서, 우리는 입력 $\mathbf{z}t$뿐만 아니라 A의 행과 열을 σ에 따라 정렬합니다. 이는 효율적으로 구현될 수 있는 고전적인 어텐션 편향(그림 7)을 만듭니다. 우리는 또한 토큰이 원래 위치 임베딩을 유지하도록 $\mathbf{z}t$의 위치 임베딩을 σ에 따라 정렬합니다. 손실을 계산할 때, 우리는 추가적으로 목표 $\mathbf{x}$를 σ에 따라 정렬합니다.

순차 단계

Eso-LM (A) 및 Eso-LM (B) 모두에서, 우리는 $\mathbf{z}0$, $\mathbf{x}$ (입력 $\mathbf{z}0 \oplus \mathbf{x}$로 연결하기 전), 그리고 A에 있는 네 개의 L×L 블록 각각의 행과 열을 σ에 따라 정렬합니다. 우리는 추가적으로 $\mathbf{z}0$의 클린 토큰에 대한 특정 추가 어텐션 연결을 추가합니다. 이는 다른 토큰이 $\mathbf{z}0$의 클린 토큰에 어텐션하지 않기 때문에 트랜스포머 출력에 영향을 미치지 않습니다. 이 두 가지 변환은 효율적으로 구현될 수 있는 희소하고 패턴화된 블록 구조(그림 8)를 가진 어텐션 편향을 만듭니다. 우리는 또한 토큰이 원래 위치 임베딩을 유지하도록 $\mathbf{z}0$과 $\mathbf{x}$의 위치 임베딩을 σ에 따라 정렬합니다. 손실을 계산할 때, 우리는 추가적으로 목표 $\mathbf{x}$를 σ에 따라 정렬합니다.

!

!

B.5.2 샘플링

그림 9는 Eso-LM (A)의 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다. 그림 10은 Eso-LM (B)의 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다.

B.6 실험 세부 사항

B.6.1 저 불일치 샘플러

훈련 중 분산을 줄이기 위해 Kingma et al.

B.6.2 우도 평가

우도를 평가하기 위해 t에 대한 단일 몬테카를로 추정치를 사용합니다. 추정치의 분산을 줄이기 위해 저 불일치 샘플러

!

!

B.6.3 언어 모델링

우리는 [15, 26]에 따라 One Billion Words 데이터셋을 토큰화 해제했습니다. 코드는 여기서 찾을 수 있습니다. 우리는 Austin et al.

우리는 GPT2 토크나이저로 OpenWebText를 토큰화했습니다. 우리는 시퀀스들을 연결하고 래핑하여 길이를 1,024로 만들었습니다. 래핑할 때, 연결된 시퀀스들 사이에 eos 토큰을 추가했습니다. One Billion Words와 달리, OpenWebText에 대한 최종 전처리된 시퀀스들은 첫 번째 및 마지막 토큰으로 특별한 토큰을 가지지 않습니다. OpenWebText는 검증 분할이 없으므로, 마지막 10만 개의 문서를 검증용으로 남겨두었습니다.

Eso-LM은 우리의 자기회귀(autoregressive) 기준선인 SEDD, MDLM, UDLM, DUO와 동일한 매개변수화를 공유합니다: Lou et al.

우리는 로그-선형 노이즈 스케줄 $\alpha t = \alpha 0(1-t)$을 사용합니다. AdamW 옵티마이저를 배치 크기 512로, 0에서 3e-4까지의 일정 학습률 웜업을 2,500 스텝 동안 사용합니다. One Billion Words에서는 1백만 스텝 동안, OpenWebText에서는 25만 스텝 동안 일정한 학습률을 사용합니다. 드롭아웃 비율은 0.1입니다. 모델은 H200 GPU에서 훈련합니다.
One Billion Words에서 1백만 스텝 훈련은 ~21일 때 α0=1시간, ~34일 때 α0<1시간이 소요됩니다.
OpenWebText에서 25만 스텝 훈련은 ~50일 때 α0=1시간, ~69일 때 α0<1시간이 소요됩니다.

부록 C 추가 실험

C.1 분할 비율에 대한 제거 연구

표 5를 참조하십시오.

표 5: LM1B에 대한 Eso-LM (A)의 테스트 퍼플렉시티(↓), 500K 스텝 훈련 vs. 훈련 중 (7)에서 MDM 손실을 평가하는 데 사용된 각 배치 내 예시의 비율. 각 배치 내 나머지 예시들은 훈련 중 (7)에서 AR 손실을 평가하는 데 사용됩니다.

	0.75	**0.5**	**0.25**	**0.125**
Eso-LM (A)				
α0=0.5	32.25	31.53	Diverged	Diverged
α0=0.25	30.49	29.33	Diverged	Diverged
α0=0.125	27.76	26.73	Diverged	Diverged
α0=0.0625	25.92	25.07	Diverged	Diverged

C.2 OWT 퍼플렉시티

표 6을 참조하십시오.

표 6: 250K 및 1M 스텝 훈련 모델의 OWT 테스트 퍼플렉시티(PPL; ↓). 확산 모델 및 보간법에 대한 범위를 보고합니다. ǂ

	PPL 250K (↓)	**PPL 1M (↓)**
Autoregressive		
Transformer	17.90$	17.54$
Diffusion		
SEDD Absorb [15]	26.81$	24.10$
MDLM [26]	25.76$	22.98$
UDLM [29]	30.52$	27.43

	PPL 250K (↓)	PPL 1M (↓)
DUO [27]	27.14$	25.20
Interpolating diffusion and autoregressive		
BD3-LMs [1]		
L' = 16	23.57†	22.27‡
Eso-LM (A) (Ours)		
α0 = 1	26.21	-
Eso-LM (B) (Ours)		
α0 = 1	30.14	-
α0 = 0.5	27.85	-
α0 = 0.25	24.73	-
α0 = 0.125	21.87	-

C.3 제로샷 우도 평가

우리는 또한 OWT에서 훈련된 모델을 가져와 보지 못한 데이터셋(표 7)을 얼마나 잘 모델링하는지 평가함으로써 모델의 일반화 능력을 탐구합니다. 우리는 Eso-LM의 퍼플렉시티를 SEDD와 비교합니다.

---

[2], MDLM [26], BD3-LMs [1], and an AR Transformer language model. Our zero-shot datasets include the validation splits of Penn Tree Bank (PTB; [16]), Wikitext [17], LM1B, Lambada [21], AG News [36], and Scientific Papers (Pubmed and Arxiv subsets; [4]).

표 7: OWT에서 250K 스텝 훈련된 모델의 제로샷 퍼플렉시티(↓). 확산 모델 및 보간법에 대한 범위를 보고합니다. AR에 대한 수치는

	PTB	Wikitext	LM1B	Lambada	AG News	Pubmed	Arxiv
AR	81.07	25.32	51.14	52.13	52.11	48.59	41.22
MDLM	93.82	36.89	69.45	53.05	67.33	49.47	43.84
SEDD Absorb	99.59	38.55	72.51	52.16	72.62	47.07	41.18
BD3-LM (L' = 16)	90.63	33.14	64.88	53.09	62.5	43.25	39.82
Eso-LM (B) (Ours)							
α0 = 1	126.29	45.08	82.01	61.37	98.22	62.37	55.76
α0 = 0.5	110.70	39.57	75.75	57.33	86.65	60.20	53.78
α0 = 0.25	105.19	37.32	67.69	60.15	75.74	62.45	55.31
α0 = 0.125	97.46	35.65	60.11	69.13	65.26	65.27	57.4

C.4 OWT에서 훈련된 모델이 생성한 샘플의 생성 퍼플렉시티 및 엔트로피

그림 3에서 NFEs를 변경하여 샘플 품질이 어떻게 변하는지 연구합니다. 생성 PPL 및 엔트로피의 수치는 표 8 (Eso-LM), 표 9 (MDLM), 표 10 (BD3-LM)에서 찾을 수 있습니다.

표 8: OWT에서 250K 스텝 훈련된 Eso-LM (B) 샘플의 생성 PPL (↓) 및 엔트로피 (↑).

α0train	**α0eval**	**T**	**NFE**	**Gen. PPL (↓)**	**Entropy (↑)**
0.125	0.0625	16	977.18	23.15	5.37
0.125	0.0625	128	1011.37	25.51	5.41
0.125	0.0625	1024	1022.13	26.85	5.43
0.125	0.25	16	806.47	46.68	5.45
0.125	0.25	128	896.41	44.19	5.42
0.125	0.25	1024	999.24	49.55	5.47
0.125	1.0	16	16.0	279.63	5.54
0.125	1.0	128	127.96	151.77	5.4
0.125	1.0	1024	647.06	159.31	5.42
0.25	0.0625	16	977.18	23.95	5.36
0.25	0.0625	128	1011.37	26.13	5.39
0.25	0.0625	1024	1022.13	27.96	5.41
0.25	0.25	16	806.47	42.31	5.43
0.25	0.25	128	896.41	38.48	5.39
0.25	0.25	1024	999.24	43.71	5.44
0.25	1.0	16	16.0	152.16	5.55
0.25	1.0	128	127.96	72.74	5.4
0.25	1.0	1024	647.06	72.36	5.42
1.0	0.0625	16	977.18	31.33	5.21
1.0	0.0625	128	1011.37	30.9	5.19
1.0	0.0625	1024	1022.13	31.11	5.19
1.0	0.25	16	806.47	50.14	5.4
1.0	0.25	128	896.41	43.59	5.33
1.0	0.25	1024	999.24	43.6	5.32
1.0	1.0	16	16.0	118.92	5.53
1.0	1.0	128	127.96	54.36	5.37
1.0	1.0	1024	647.06	49.4	5.33

표 9: OWT에서 1M 스텝 훈련된 MDLM 샘플의 생성 PPL (↓) 및 엔트로피 (↑).

T	**NFE**	**Gen. PPL (↓)**	**Entropy (↑)**
16	16	148.08	5.61
128	127.96	48.66	5.43

T	NFE	Gen. PPL (↓)	Entropy (↑)
256	251.35	45.07	5.4
512	442.87	42.58	5.37
1024	647.91	41.92	5.36
4096	906.32	41.53	5.36

표 10: OWT에서 1M 스텝 훈련된 BD3-LM 샘플의 생성 PPL (↓) 및 엔트로피 (↑).

Block size	T	NFE	Gen. PPL (↓)	Entropy (↑)
4	4096	925.32	41.04	5.31
4	2048	830.93	88.4	5.19
4	1024	672.91	199.99	4.79
4	512	480.21	186.05	4.23
4	256	256.0	199.22	4.39
8	4096	915.14	39.88	5.33
8	2048	814.07	59.84	5.34
8	1024	646.69	137.99	5.24
8	512	439.32	258.95	4.92
8	256	254.9	278.57	4.39
8	128	128	466.1	4.91
16	4096	910.37	38.74	5.33
16	2048	811.01	47.02	5.34
16	1024	642.25	73.84	5.36
16	512	427.46	189.26	5.33
16	256	245.77	351.63	5.1
16	128	128	447.24	4.61